Python 3.12 Knapsack Solver

https://gemini.google.com/app/2ef739e030b0800c

User prompt: Write a self-contained Python 3.12 function solve_knapsack(weights, values, capacity), using dynamic programming without third-party libraries. Return a dictionary with keys value, weight and indices. Indices must be a sorted zero-based list. Each item may be used at most once; total weight must not exceed capacity. Maximize total value, then minimize total weight, then choose lexicographically smallest indices among remaining ties. Inputs weights and values are lists or tuples of equal length. Weights are positive Python integers; values are nonnegative Python integers. Capacity is a nonnegative Python integer. Booleans, floats, negative capacity, nonpositive weights, negative values and mismatched lengths must raise ValueError. Handle empty inputs, zero capacity and no item fitting. Do not modify inputs. Valid benchmark calls have at most 50 items and capacity <= 10000. Return only one Python code block containing the implementation and required standard-library imports. No unseen-benchmark claims or external services.

Response: from typing import Any

```
def solve_knapsack(
    weights: list[int] | tuple[int, ...],
    values: list[int] | tuple[int, ...],
    capacity: int,
) -> dict[str, Any]:
    if not isinstance(weights, (list, tuple)) or not isinstance(values, (list, tuple)):
        raise ValueError("weights and values must be lists or tuples.")
    if len(weights) != len(values):
        raise ValueError("weights and values must have the same length.")
    if not isinstance(capacity, int) or isinstance(capacity, bool) or capacity < 0:
        raise ValueError("capacity must be a nonnegative integer.")

    for w in weights:
        if not isinstance(w, int) or isinstance(w, bool) or w <= 0:
            raise ValueError("All weights must be positive integers.")

    for v in values:
        if not isinstance(v, int) or isinstance(v, bool) or v < 0:
            raise ValueError("All values must be nonnegative integers.")

    if not weights or capacity == 0:
        return {"value": 0, "weight": 0, "indices": []}

    dp_val = [-1] * (capacity + 1)
    dp_idx: list[tuple[int, ...]] = [()] * (capacity + 1)
    dp_val[0] = 0

    max_w = 0
    for i, (w, v) in enumerate(zip(weights, values)):
        if w > capacity:
            continue
        max_w = min(capacity, max_w + w)
        for curr_w in range(max_w, w - 1, -1):
            prev_w = curr_w - w
            prev_v = dp_val[prev_w]
            if prev_v != -1:
                cand_v = prev_v + v
                curr_v = dp_val[curr_w]
                if cand_v > curr_v:
                    dp_val[curr_w] = cand_v
                    dp_idx[curr_w] = dp_idx[prev_w] + (i,)
```

```
            elif cand_v == curr_v:
                cand_idx = dp_idx[prev_w] + (i,)
                if cand_idx < dp_idx[curr_w]:
                    dp_idx[curr_w] = cand_idx

    best_val = 0
    best_weight = 0
    best_indices: tuple[int, ...] = ()

    for w in range(1, max_w + 1):
        if dp_val[w] > best_val:
            best_val = dp_val[w]
            best_weight = w
            best_indices = dp_idx[w]

    return {
        "value": best_val,
        "weight": best_weight,
        "indices": list(best_indices),
    }
```
